

Commande de vol latéral

alazard@isae.fr

BERGE Florent

GROUSSEAU Léa

KHERFOUCHE--BERNAUDAT Gaël

Description du modèle d'état:

- $x = [\beta; p; r; \phi]$
- $u = [dp; dr]$
- $y = [n_y; p; r; \phi]$

```
a=[ -0.1400    0.0530   -0.9990    0.0470;
    -2.4610   -0.9920    0.2620     0;
     1.5950   -0.0410   -0.2670     0;
      0      1.0000    0.0530     0];
b=[0 0.03;0.404  0.26;0.00 -0.68;0 0];
c=[ 0.0433   -0.0003    0.0016     0;...
    0      1.0000     0         0;...
    0         0      1.0000     0 ;...
    0         0         0      1.0000];
d=[0.0001   -0.0075;0 0;0 0;0 0];
plant=ss(a,b,c,d);
plant.u = {'dp','dr'};
plant.StateName = {'beta', 'p', 'r', 'phi'};
```

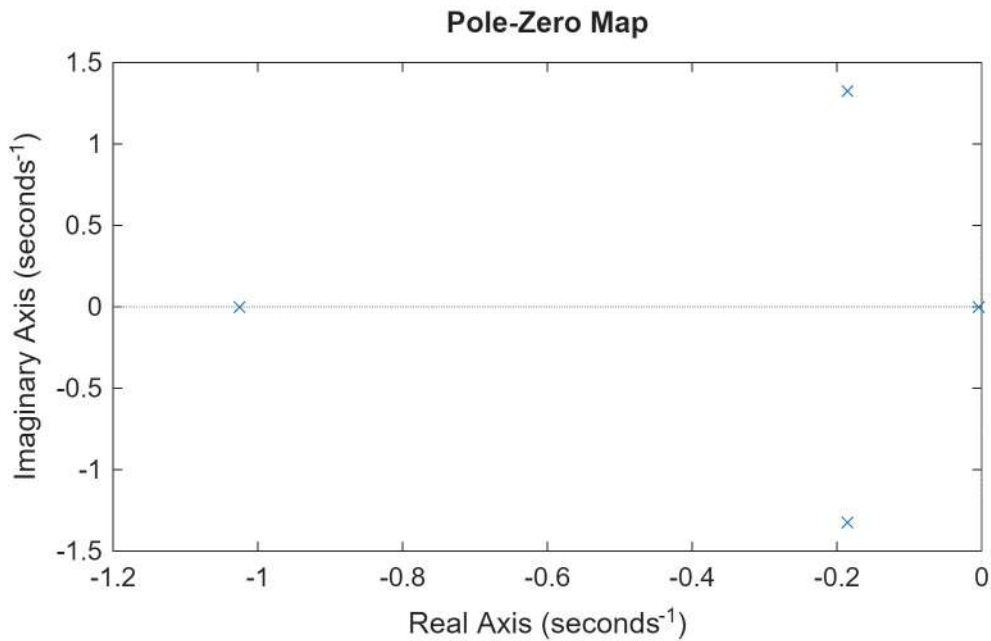
Analyse en boucle ouverte

Analyse de la dynamique:

```
damp(plant)
```

Pole	Damping	Frequency (rad/seconds)	Time Constant (seconds)
-3.84e-03	1.00e+00	3.84e-03	2.60e+02
-1.03e+00	1.00e+00	1.03e+00	9.75e-01
-1.85e-01 + 1.33e+00i	1.38e-01	1.34e+00	5.41e+00
-1.85e-01 - 1.33e+00i	1.38e-01	1.34e+00	5.41e+00

```
figure
pzmap(plant)
```

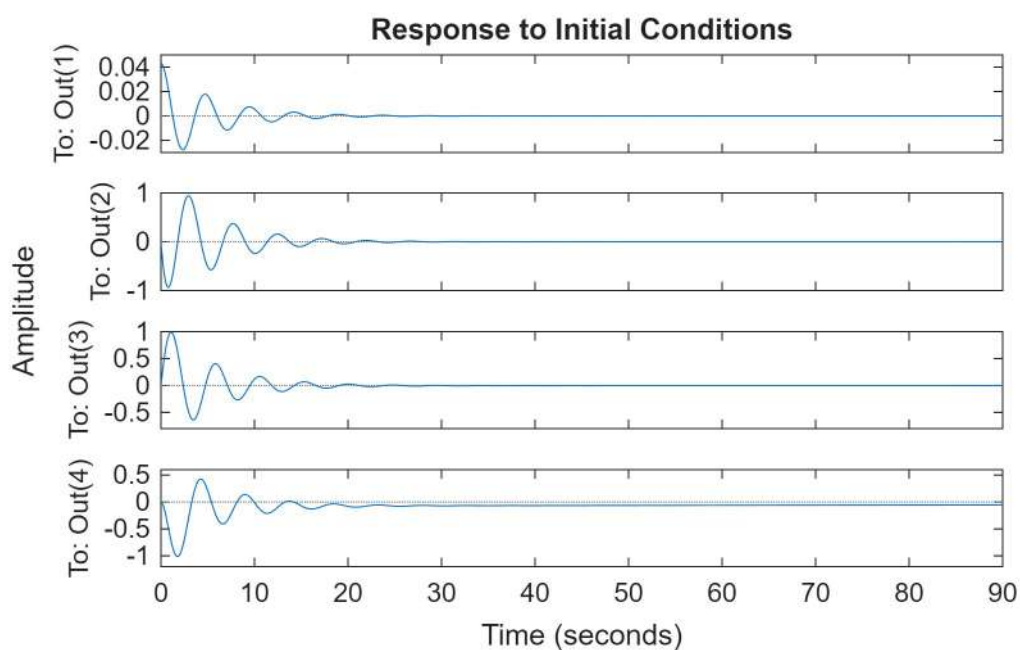


- En $[0;-1]$: mode de roulis pur
- En $[0;0]$: mode spirale
- Les deux pôles complexes conjugués: mode de roulis hollandais (mal amorti)

On observe que tous les poles sont à partie réelle négative donc le système est stable en BO mais le faible amortissement du roulis hollandais et la lenteur du mode spirale sont problématique pour le pilotage

Réponse du système à une condition initiale sur β :

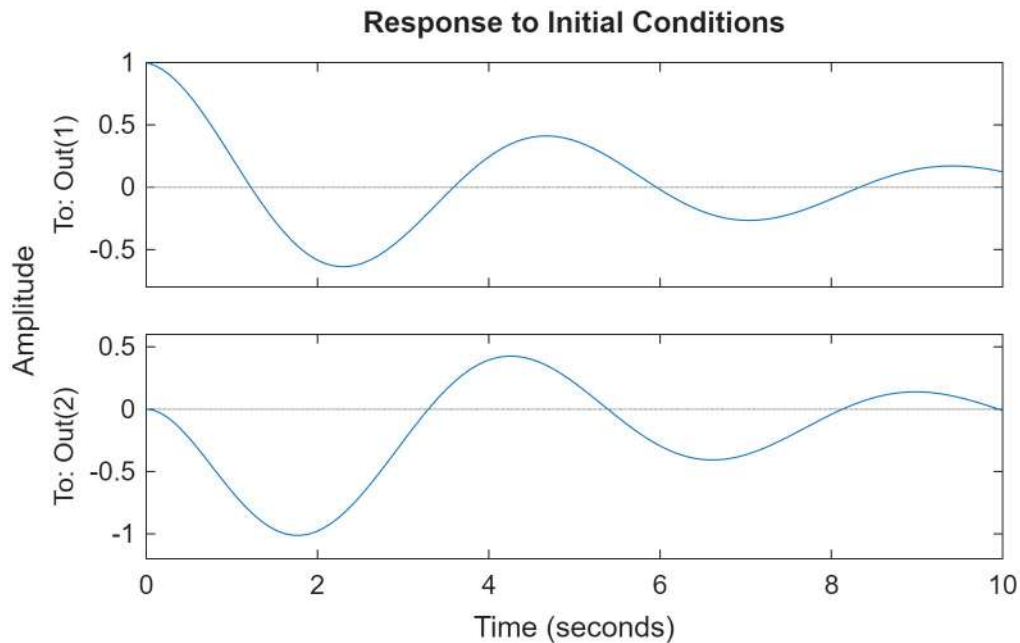
```
x0 = [1;0;0;0];
figure
initial(plant, x0)
```



On observe que les oscillations sont longues (roulis hollandais) couplé à une dérive lente sur ϕ .

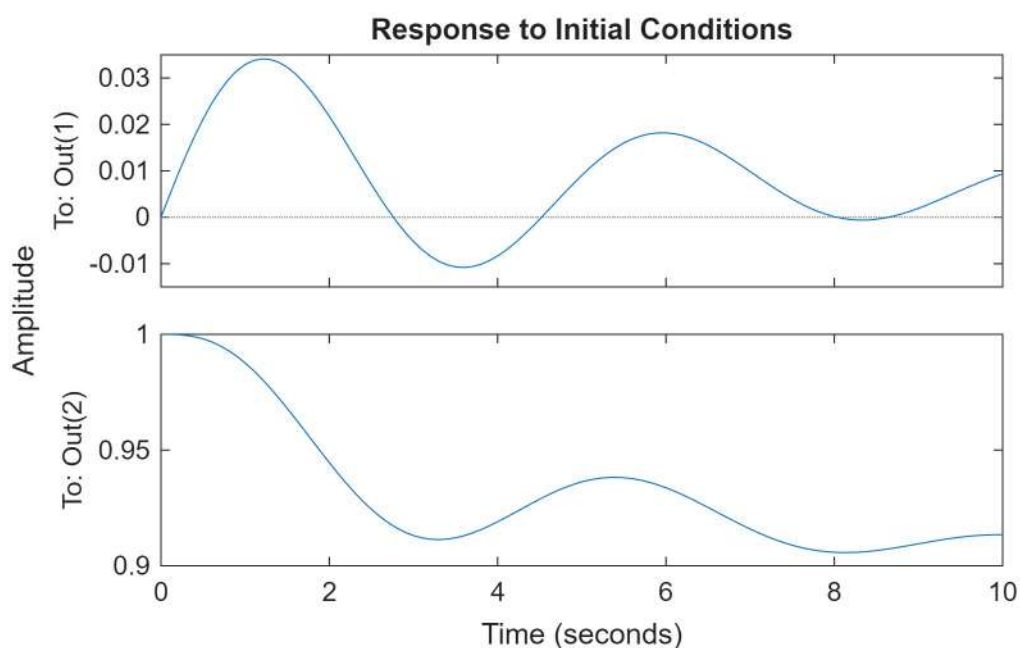
Réponse de β et ϕ à des conditions initiales sur β et ϕ sur un horizon de 10 s:

```
figure
initial(ss(plant.a, plant.b, [1 0 0 0; 0 0 0 1],0), [1;0;0;0], 10) % C0 sur beta
```



On observe que la condition initiale sur β entraîne des oscillations sur β ET ϕ . Les deux états sont donc couplés.

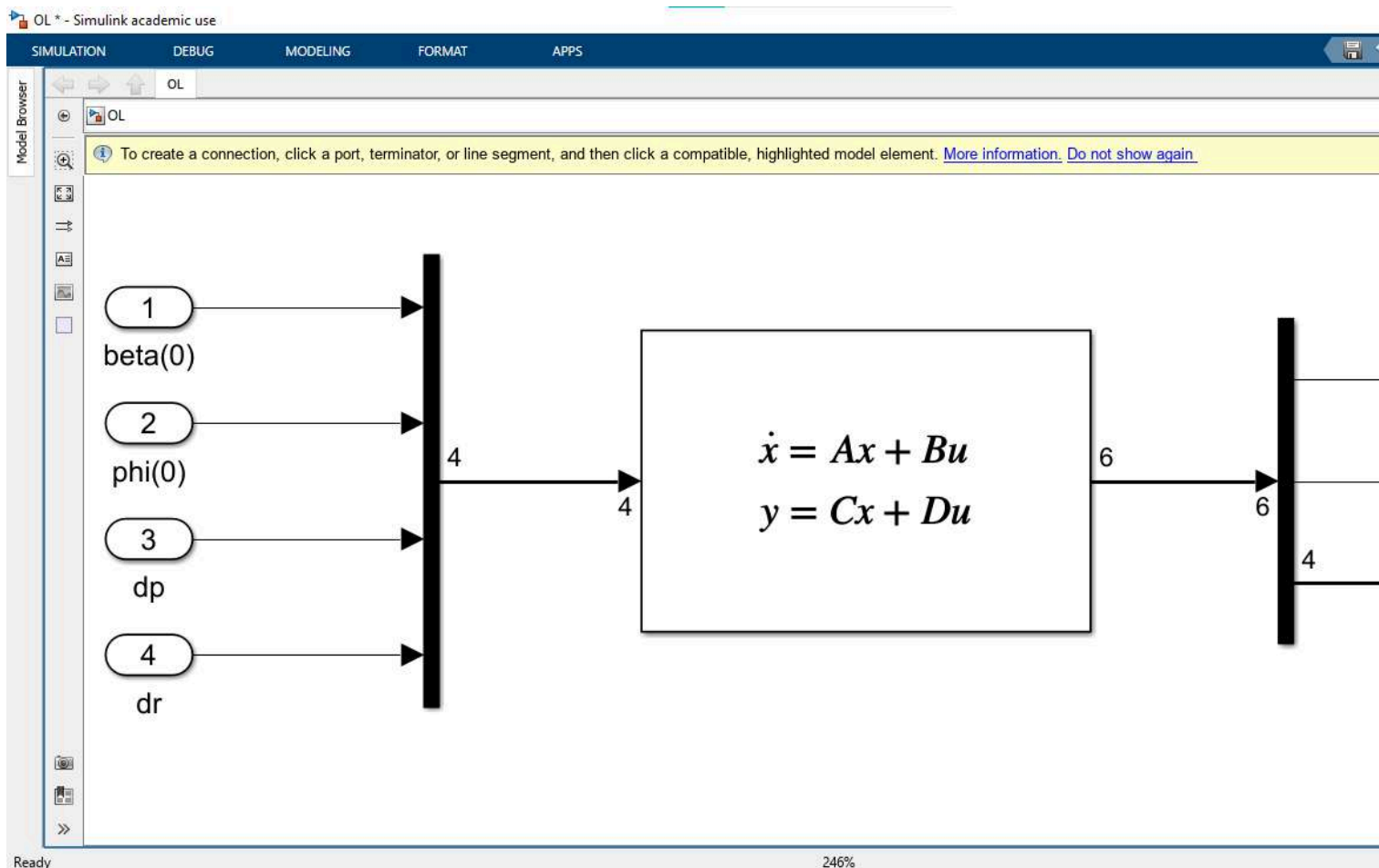
```
figure
initial(ss(plant.a, plant.b, [1 0 0 0; 0 0 0 1],0), [0;0;0;1], 10) % C0 sur phi
```



De même, on observe que la condition initiale sur ϕ entraîne des oscillations sur β ET ϕ , et un très faible amortissement (on passe de 1 à 0.9 en 10 s). Les deux états sont donc couplés.

L'utilisation de la fonction *initial* suppose que l'on connaît l'état du système, ce qui peut être laborieux quand on va reboucler des filtres et des correcteurs dynamiques. On préfère utiliser un schéma simulink avec 2 entrées fictives dont les réponses impulsionnelles correspondent aux réponses à des conditions initiales sur β et ϕ .

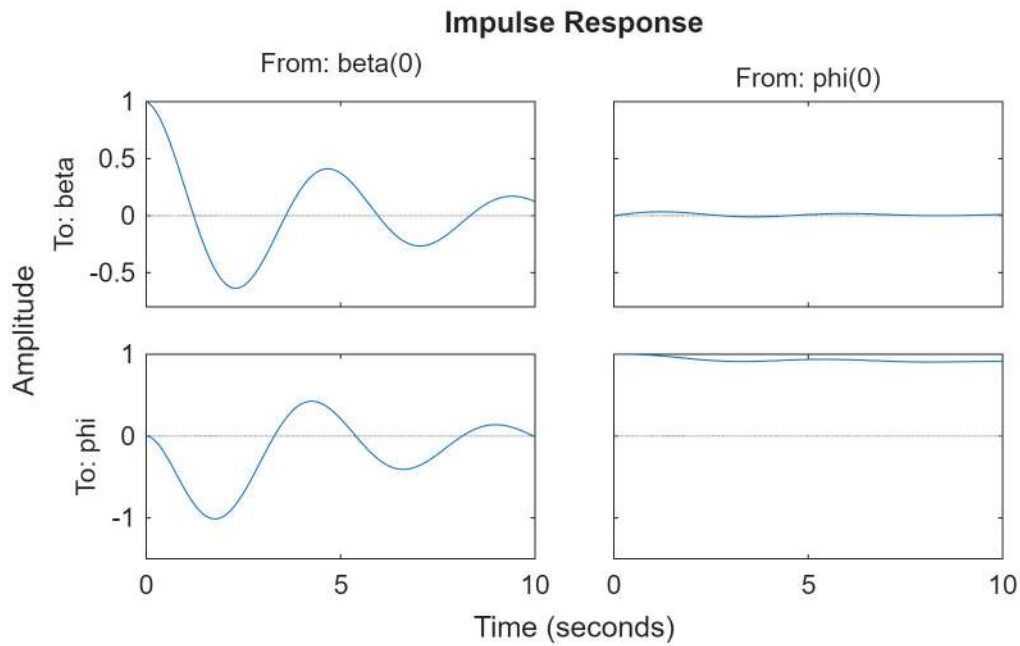
D'où le schéma simulink:



```
B0 = linearize('OL');  
figure
```

Réponse impulsionnelle:

```
impulse(B0(1:2,1:2),10) % on regarde les sorties 1 et 2 (beta, hi) pour une  
impulsion en entrée 1 et 2 (beta(0), phi(0))
```



Commande modale (placement de pôles)

Commande modale non découplante

```
lda = [-1.1; -1; -1+1.3i; -1-1.3i]; % Placement des pôles
Kx = place(plant.a,plant.b,lda);
K = Kx*inv(plant.c-plant.d*Kx) % u = K*y
```

```
K = 2x4
-167.7677    7.9423    0.5393    7.9425
-26.2268    8.6304   -2.2536    8.5835
```

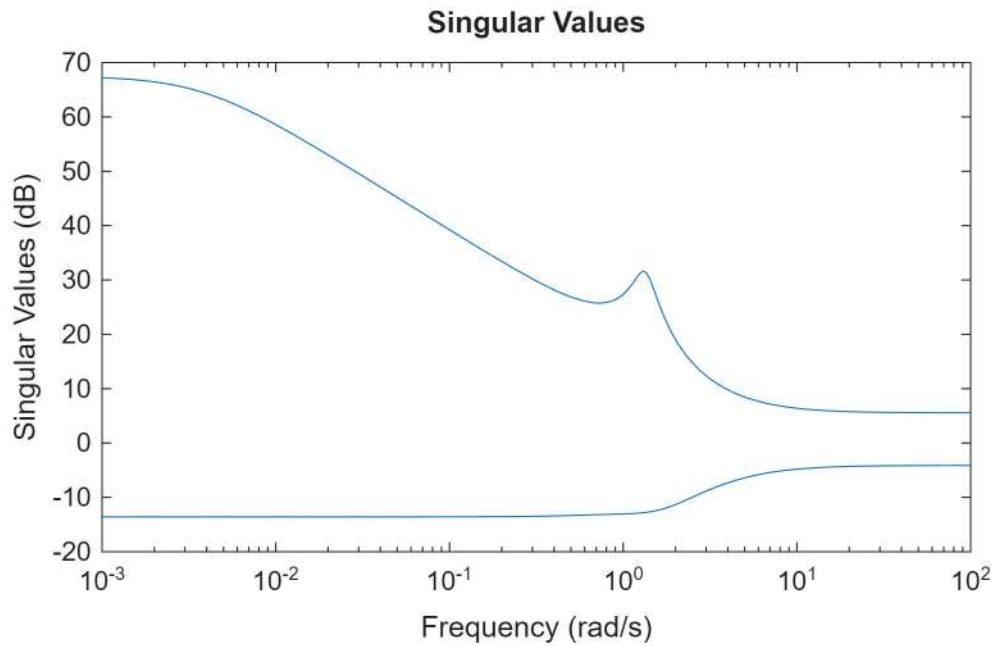
Vérification:

```
damp(feedback(plant, K))
```

Pole	Damping	Frequency (rad/seconds)	Time Constant (seconds)
-1.00e+00	1.00e+00	1.00e+00	1.00e+00
-1.10e+00	1.00e+00	1.10e+00	9.09e-01
-1.00e+00 + 1.30e+00i	6.10e-01	1.64e+00	1.00e+00
-1.00e+00 - 1.30e+00i	6.10e-01	1.64e+00	1.00e+00

Marge de module:

```
figure
sigma(eye(2)+K*plant)
```



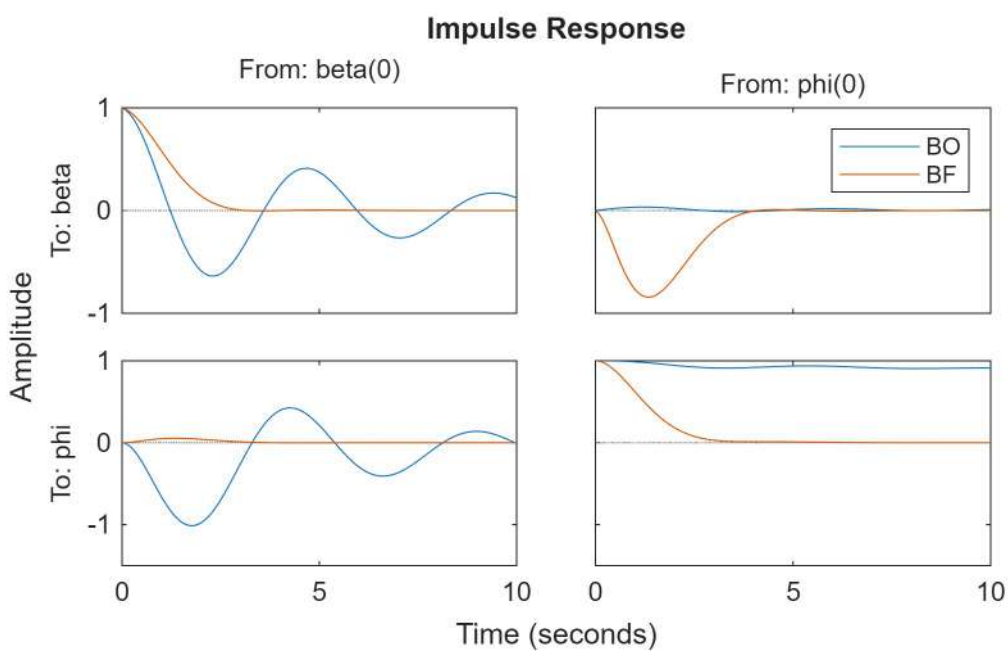
```
S=inv(eye(2)+K*plant);
r=1/norm(S, 'inf')
```

```
r =
0.2090
```

La marge de module est très faible (<0.5), ce qui rend le système très sensible aux incertitudes de modélisation.

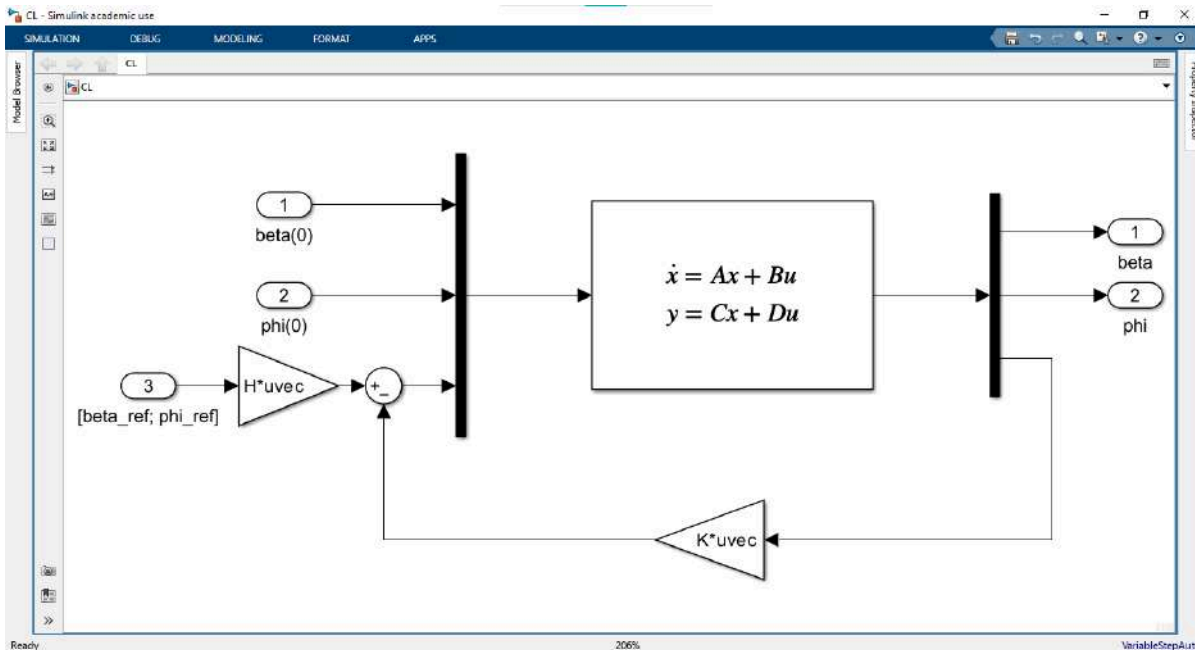
Comparaison boucle ouverte et boucle fermée

```
BF = lft(B0, -K);
figure
impz(B0(1:2,1:2),BF,10)
legend("BO", "BF")
```

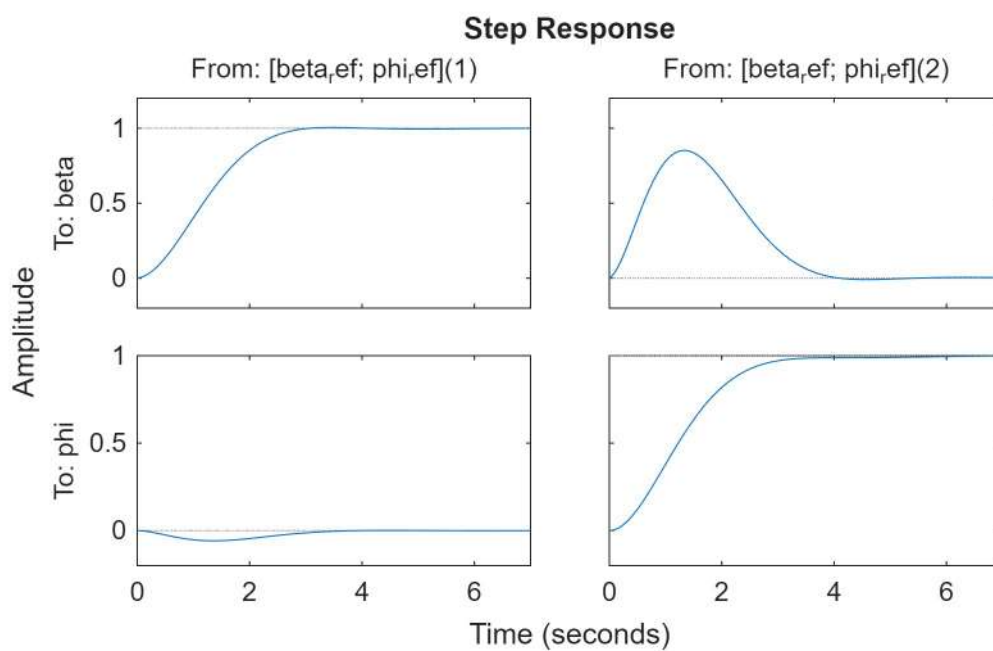


La réponse à une impulsion ϕ en boucle fermée a une influence sur β . Les modes ne sont pas découplés. On observe cependant que le gain K obtenu stabilise bien le système.

Calcul de la précommande H à partir du système en boucle fermée pour avoir un gain statique unitaire.



```
H = eye(2);
Gbf = linearize('CL1');
% figure
% step(Gbf(1:2,3:4))
H = inv(dcgain(Gbf(1:2,3:4)));
Gbf = linearize('CL1');
figure
step(Gbf(1:2,3:4))
```



La précommande H assure le gain statique unitaire du système, β et ϕ tendent vers leurs valeurs de référence en régime permanent après une impulsion.

Un couplage transitoire est cependant toujours présent.

Commande modale découplante

On impose ici les vecteurs propres associés aux valeurs propres précédentes, les modes réelles sont découplés de β et les modes complexes de ϕ .

```
v1w1 = null([plant.a-lda(1)*eye(4) plant.b; 1 0 0 0 0 0]);
v2w2 = null([plant.a-lda(2)*eye(4) plant.b; 1 0 0 0 0 0]);
v3w3 = null([plant.a-lda(3)*eye(4) plant.b; 0 0 0 1 0 0]);
v4w4 = null([plant.a-lda(4)*eye(4) plant.b; 0 0 0 1 0 0]);
V = [v1w1(1:4) v2w2(1:4) v3w3(1:4) v4w4(1:4)];
W = [v1w1(5:6) v2w2(5:6) v3w3(5:6) v4w4(5:6)];
Kx = -W*inv(V);
Kx = real(Kx);
K = Kx*inv(plant.c-plant.d*Kx);
```

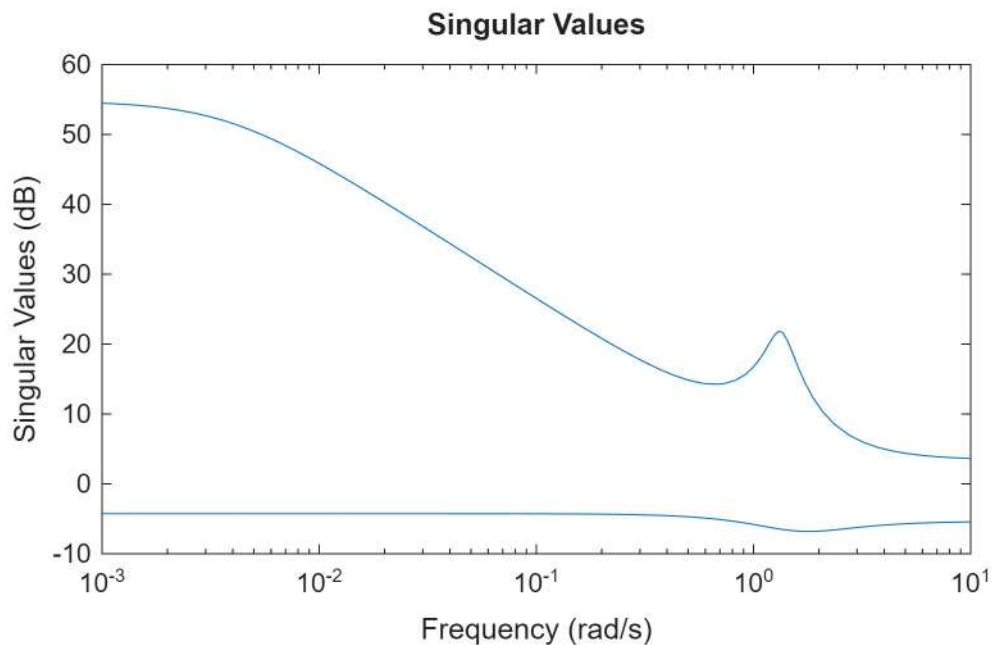
Vérification

```
damp(feedback(plant, K))
```

Pole	Damping	Frequency (rad/seconds)	Time Constant (seconds)
-1.00e+00	1.00e+00	1.00e+00	1.00e+00
-1.10e+00	1.00e+00	1.10e+00	9.09e-01
-1.00e+00 + 1.30e+00i	6.10e-01	1.64e+00	1.00e+00
-1.00e+00 - 1.30e+00i	6.10e-01	1.64e+00	1.00e+00

Marge de module

```
figure
sigma(eye(2)+K*plant)
```

```
S=inv(eye(2)+K*plant);
r=1/norm(S,'inf')
```

```
r =
0.4564
```

Comparaison boucle ouverte et boucle fermée

```
BF = lft(B0,-K)
```

```
BF =
```

```
A =
```

	State-Space(	State-Space(	State-Space(	State-Space(
State-Space(	-0.1812	0.04969	-0.9306	0.04566
State-Space(	-0.134	-2.102	-0.015	-1.102
State-Space(	2.529	0.03395	-1.817	0.03041
State-Space(	0	1	0.053	0

```
B =
```

	beta(0)	phi(0)
State-Space(	1	0
State-Space(	0	0
State-Space(	0	0
State-Space(	0	1

```
C =
```

	State-Space(	State-Space(	State-Space(	State-Space(
beta	1	0	0	0
phi	0	0	0	1

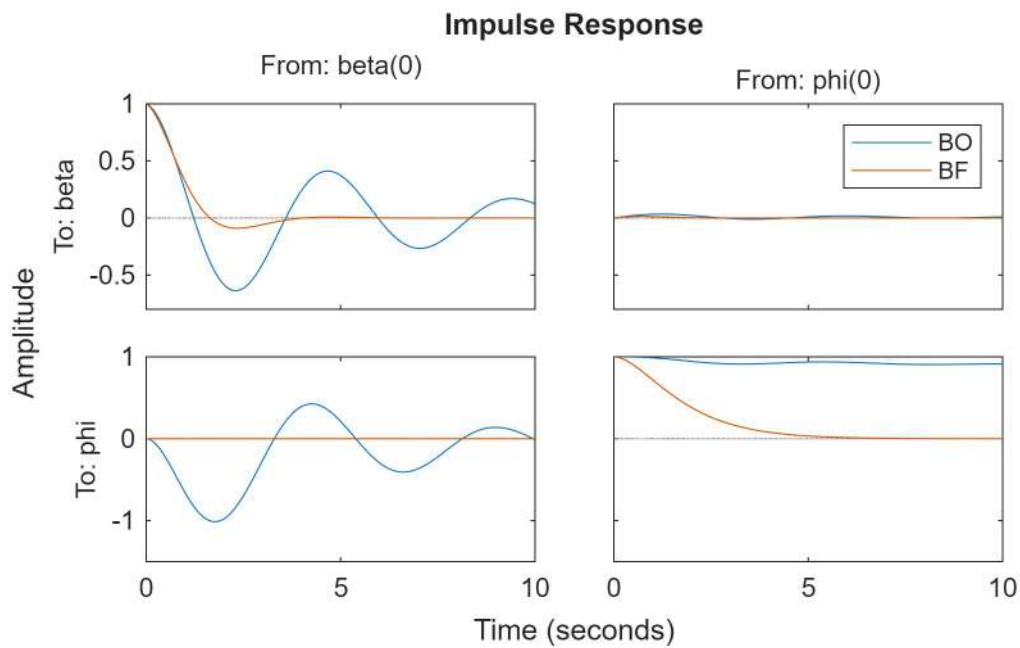
```
D =
```

	beta(0)	phi(0)
beta	0	0
phi	0	0

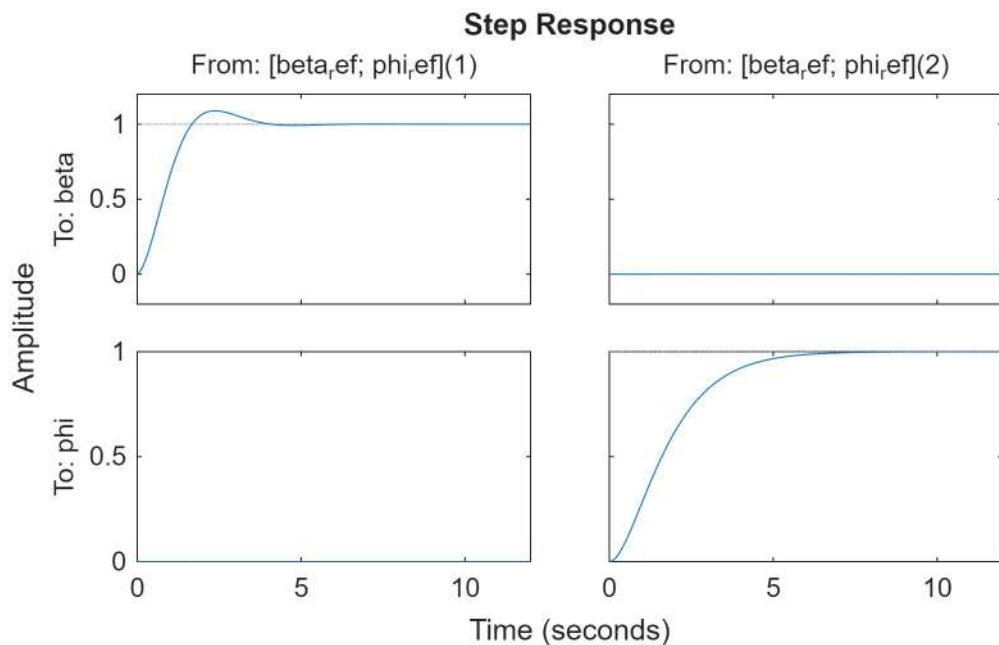
Continuous-time state-space model.
Model Properties

```
figure
impulse(B0(1:2,1:2),BF,10)
```

```
legend("BO", "BF")
```



```
H = eye(2);
Gbf = linearize('CL1');
% figure
% step(Gbf(1:2,3:4))
H = inv(dcgain(Gbf(1:2,3:4)));
Gbf = linearize('CL1');
figure
step(Gbf(1:2,3:4))
```



On observe que les réponses indicielles et impulsionnelles sont désormais découplées, les deux valeurs tendent respectivement vers leur valeur de référence mais avec un léger dépassement pour β . L'amortissement ξ est donc inférieur à 1.

Commande LQ

On cherche à minimiser $J = \int_0^\infty x^T Q_x x + u^T R u dt$

On ne va pondérer que r et

```
qr=7;qphi=2;
Qx = diag([0 0 qr qphi]);
R = eye(2);
[Kx,Px,dyn] = lqr(plant.a,plant.b,Qx,R);
damp(dyn)
```

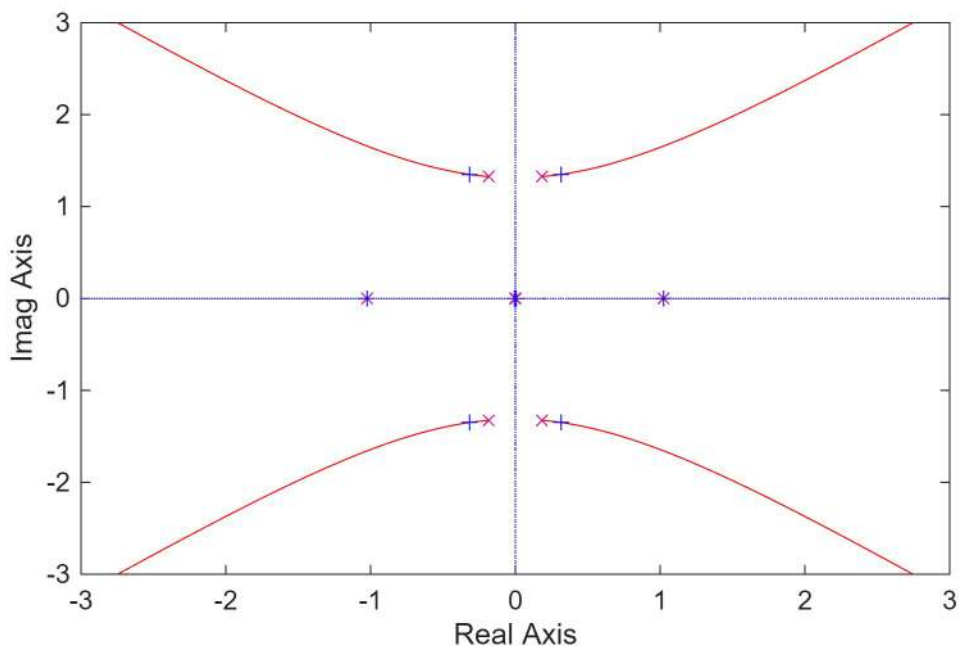
Pole	Damping	Frequency (rad/TimeUnit)	Time Constant (TimeUnit)
-7.95e-01 + 1.05e+00i	6.02e-01	1.32e+00	1.26e+00
-7.95e-01 - 1.05e+00i	6.02e-01	1.32e+00	1.26e+00
-1.02e+00 + 3.74e-01i	9.39e-01	1.09e+00	9.81e-01
-1.02e+00 - 3.74e-01i	9.39e-01	1.09e+00	9.81e-01

Pourquoi choisit-on de ne pondérer que r et ϕ ?

Observons l'évolution de la dynamique en boucle fermée en fonction de ρ lorsque l'on pondère β seul:

$$J = \int_0^\infty \beta^2 + \rho(dp^2 + dr^2)dt$$

```
addpath("bib1\");
figure
rsqlocus(plant.a,plant.b,diag([1 0 0 0]), eye(2)); % ponderer beta seul
```

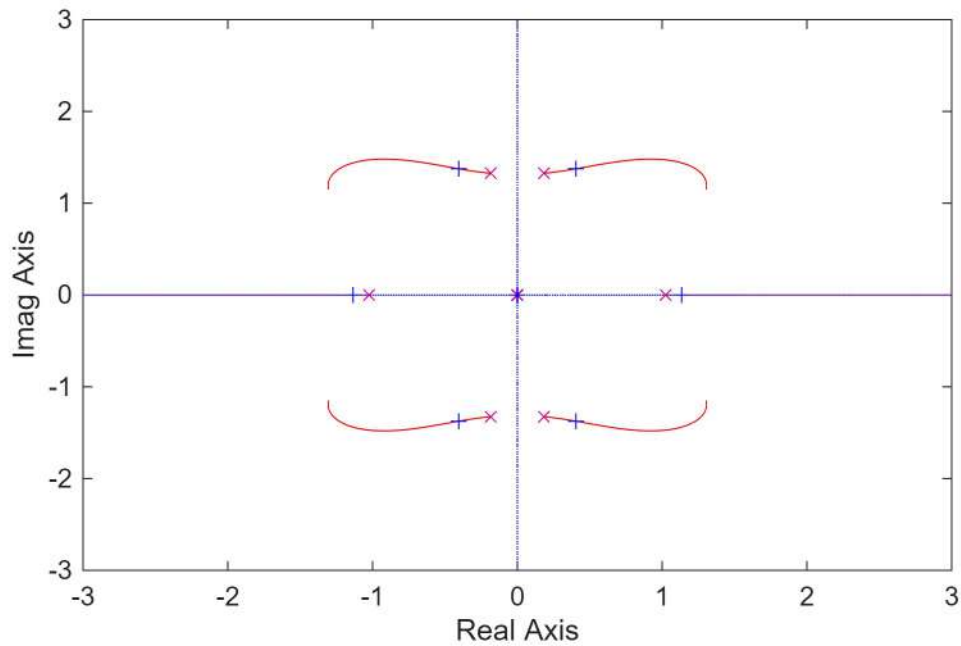


Quelque soit le réglage de ρ , il n'est pas possible de modifier les modes spirale et roulis pur.

Lorsque l'on pondère p seul:

$$J = \int_0^{\infty} p^2 + \rho(dp^2 + dr^2)dt$$

```
figure
rsqlocus(plant.a,plant.b,diag([0 1 0 0]), eye(2));
```

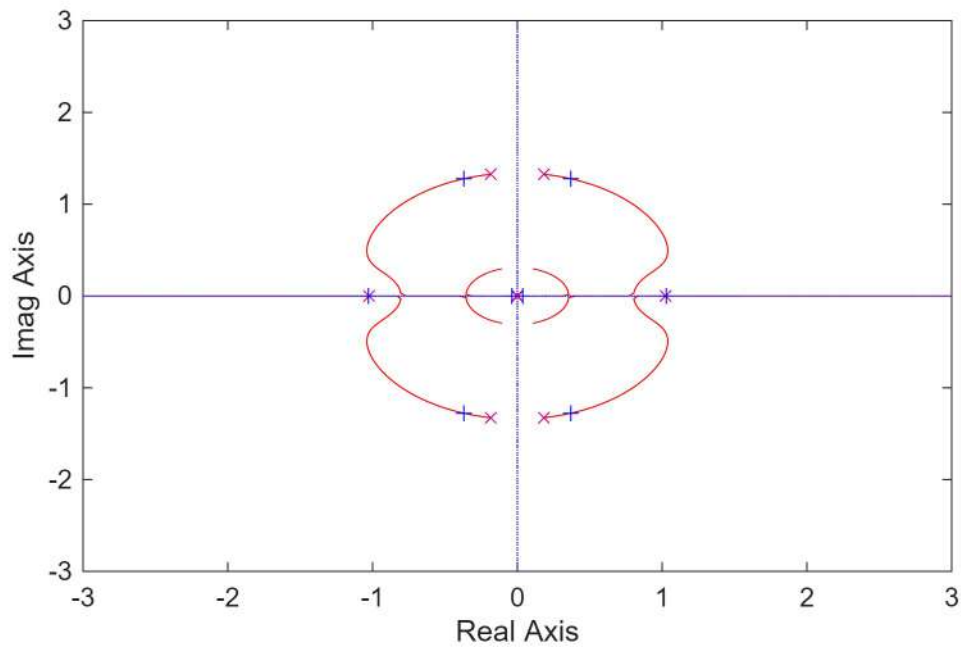


De même il n'est pas possible d'accélérer le mode spirale.

Lorsque l'on pondère r seul:

$$J = \int_0^{\infty} r^2 + \rho(dp^2 + dr^2)dt$$

```
figure
rsqlocus(plant.a,plant.b,diag([0 0 1 0]), eye(2));
```

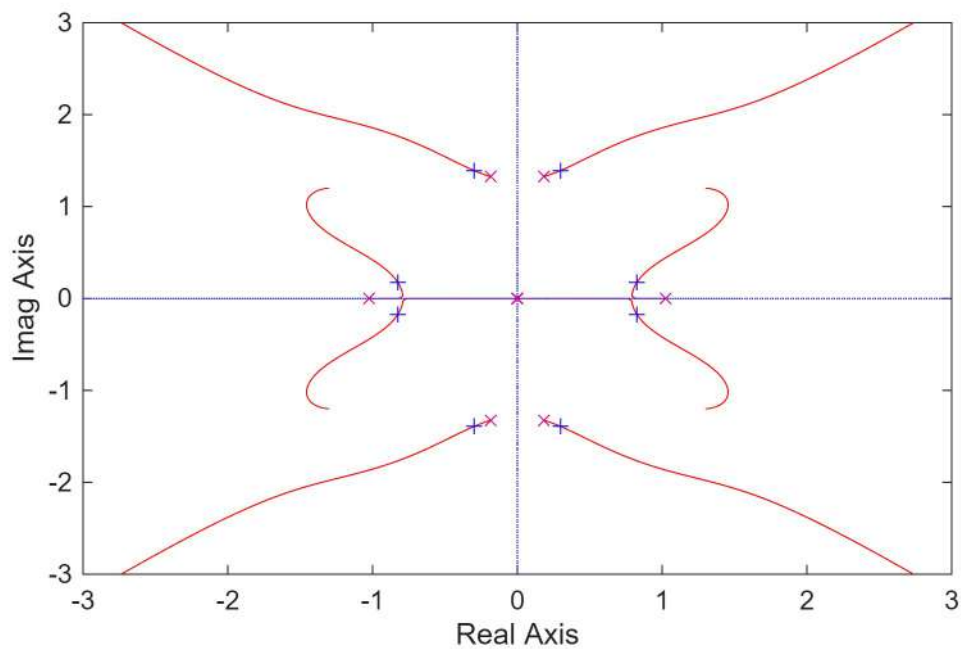


Le réglage de ρ permet d'amortir le roulis hollandais.

Lorsque l'on pondère ϕ seul:

$$J = \int_0^{\infty} \phi^2 + \rho(dp^2 + dr^2)dt$$

```
figure
rsqlocus(plant.a,plant.b,diag([0 0 0 1]), eye(2));
```

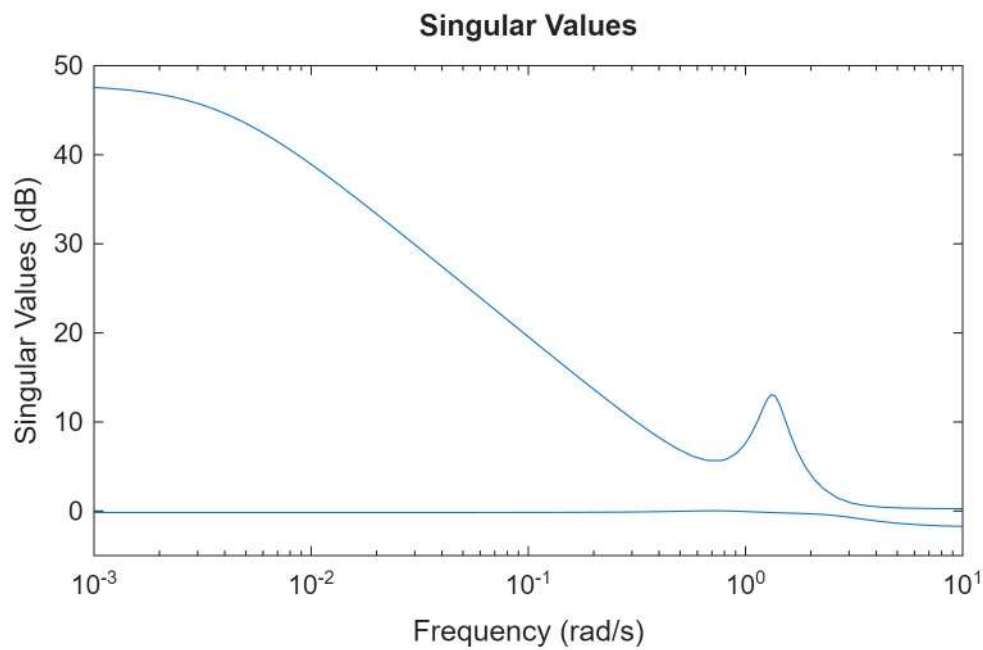


Ce réglage permet d'accélérer le mode spirale.

```
K = Kx*inv(plant.c-plant.d*Kx);
```

Marge de module

```
figure  
sigma(eye(2)+K*plant)
```

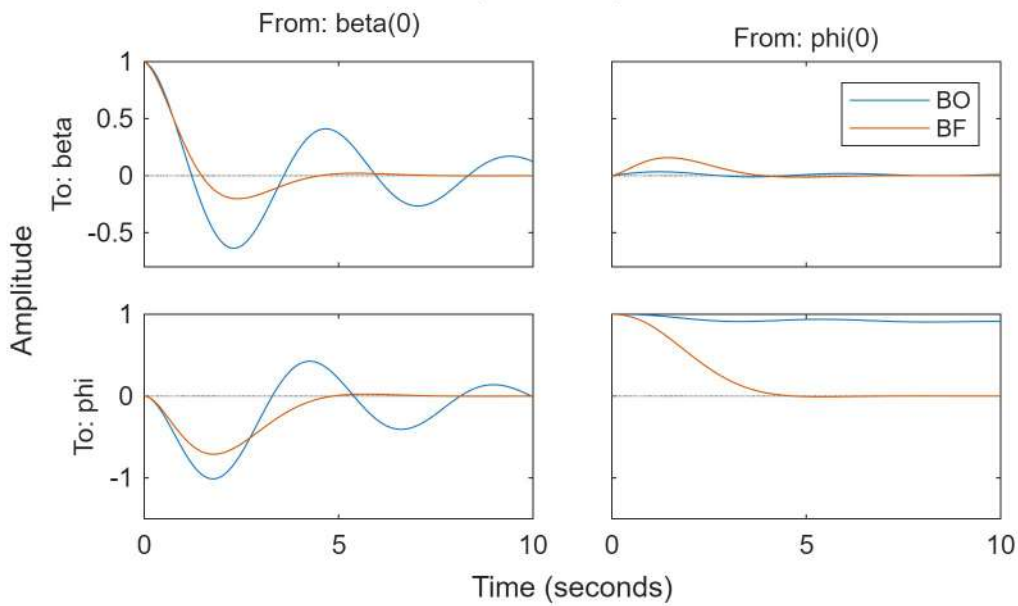


```
S=inv(eye(2)+K*plant);  
r=1/norm(S,'inf') % 0.8, pas égale à 1 car retour de sortie; Si retour d'état  
r>=1;
```

```
r =  
0.8084
```

```
BF = lft(B0,-K);  
figure  
impz(B0(1:2,1:2),BF,10)  
legend("B0","BF")
```

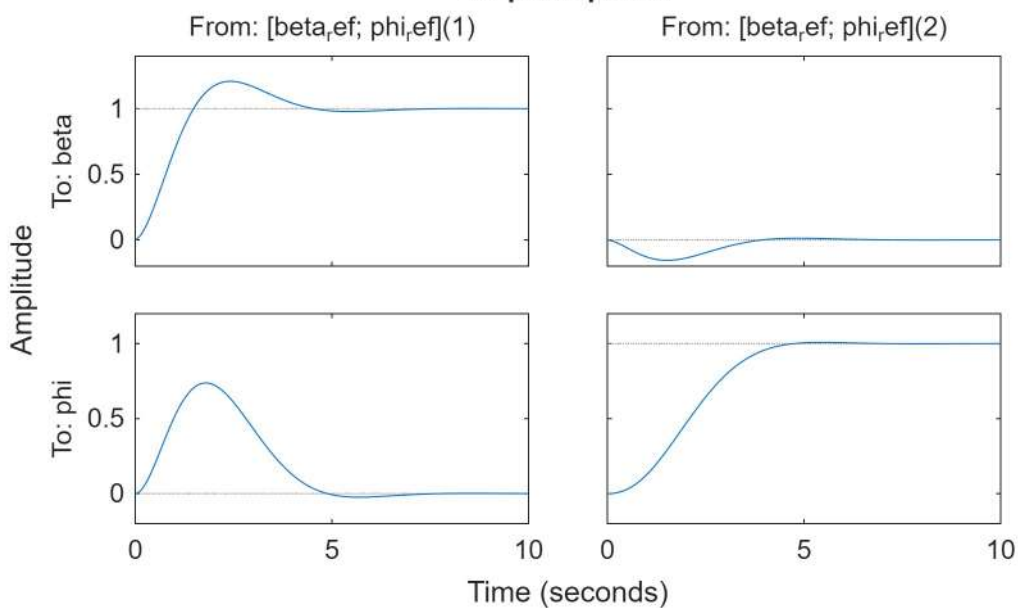
Impulse Response



Gain statique

```
H = eye(2);
Gb = linearize('CL1');
% figure
% step(Gb(1:2,3:4))
H = inv(dcgain(Gb(1:2,3:4)));
Gb = linearize('CL1');
figure
step(Gb(1:2,3:4))
```

Step Response



Cependant, cette méthode ne permet pas de découpler les modes comme le montre ces réponses indicielles.

Commande LQ avec modèle de référence implicite

On souhaite un découplage parfait, soit:

- Dynamique de lacet: $\ddot{\beta} + 2\xi_l\omega_l\dot{\beta} + \omega_l^2\beta$
- Dynamique de roulis: $\ddot{\phi} + 2\xi_r\omega_r\dot{\phi} + \omega_r^2\phi$

On rappelle $\dot{\beta} = -r$ et $\dot{\phi} = p$.

Il suit:

$$\dot{x} = A_d x$$

```
wr = 1.049;
wl = 1.64;
xir = 1.001;
xil = 0.61;
Ad = [0 0 -1 0; 0 -2*xir*wr 0 -wr^2; wl^2 0 -2*xil*wl 0; 0 1 0 0];
damp(Ad)
```

Pole	Damping	Frequency (rad/TimeUnit)	Time Constant (TimeUnit)
-1.00e+00 + 1.30e+00i	6.10e-01	1.64e+00	1.00e+00
-1.00e+00 - 1.30e+00i	6.10e-01	1.64e+00	1.00e+00
-1.10e+00	1.00e+00	1.10e+00	9.12e-01
-1.00e+00	1.00e+00	1.00e+00	9.97e-01

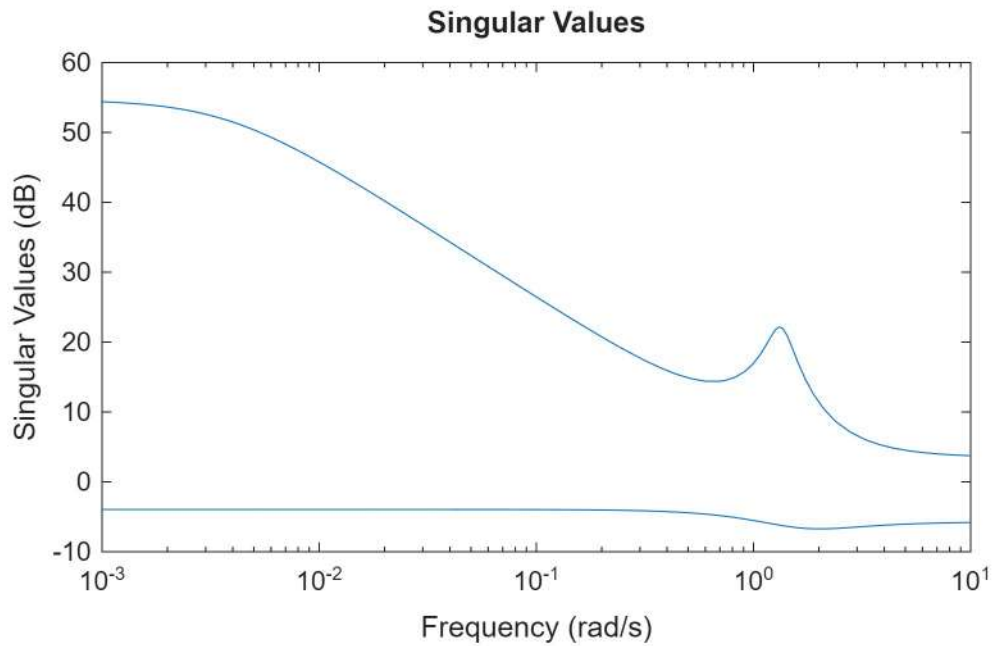
```
Qx=(plant.a-Ad)'*(plant.a-Ad);
R=(plant.b'*plant.b);
S=(plant.a-Ad)'*plant.b;
[Kx,Px,dyn] = lqr(plant.a, plant.b, Qx, R, S);
damp(dyn)
```

Pole	Damping	Frequency (rad/TimeUnit)	Time Constant (TimeUnit)
-1.01e+00	1.00e+00	1.01e+00	9.93e-01
-1.09e+00	1.00e+00	1.09e+00	9.17e-01
-1.10e+00 + 1.29e+00i	6.49e-01	1.69e+00	9.11e-01
-1.10e+00 - 1.29e+00i	6.49e-01	1.69e+00	9.11e-01

```
K = Kx*inv(plant.c-plant.d*Kx);
```

Marge de module

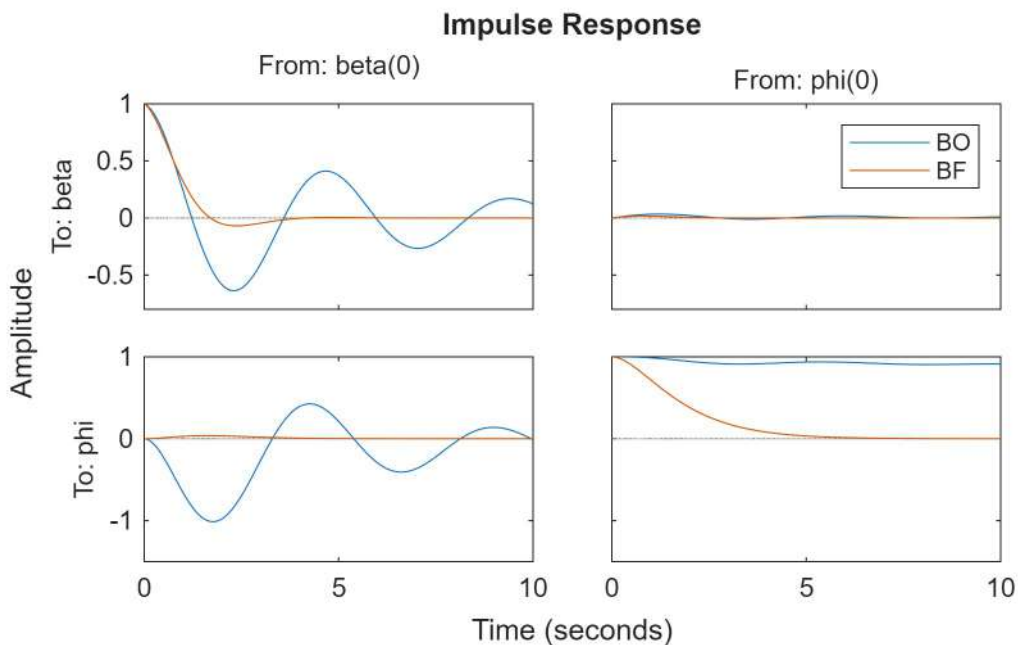
```
figure
sigma(eye(2)+K*plant)
```

```
S=inv(eye(2)+K*plant);
r=1/norm(S, 'inf')
```

```
r =
0.4607
```

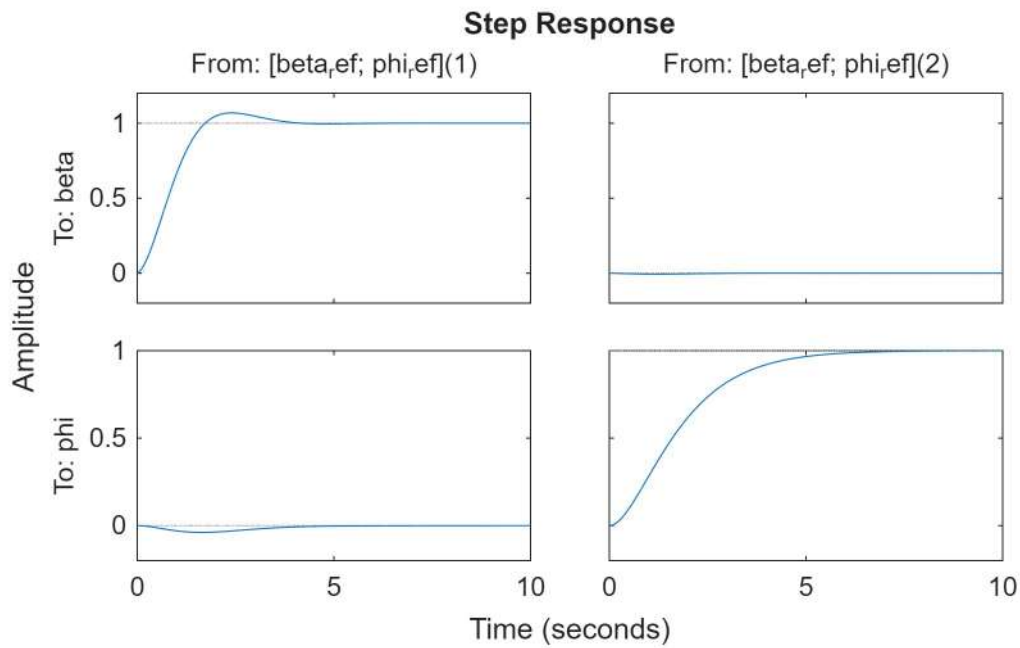
```
BF = lft(B0, -K);
figure
impz(B0(1:2,1:2),BF,10)
legend("B0", "BF")
```



Gain statique

```
H = eye(2);
Gb = linearize('CL1');
% figure
```

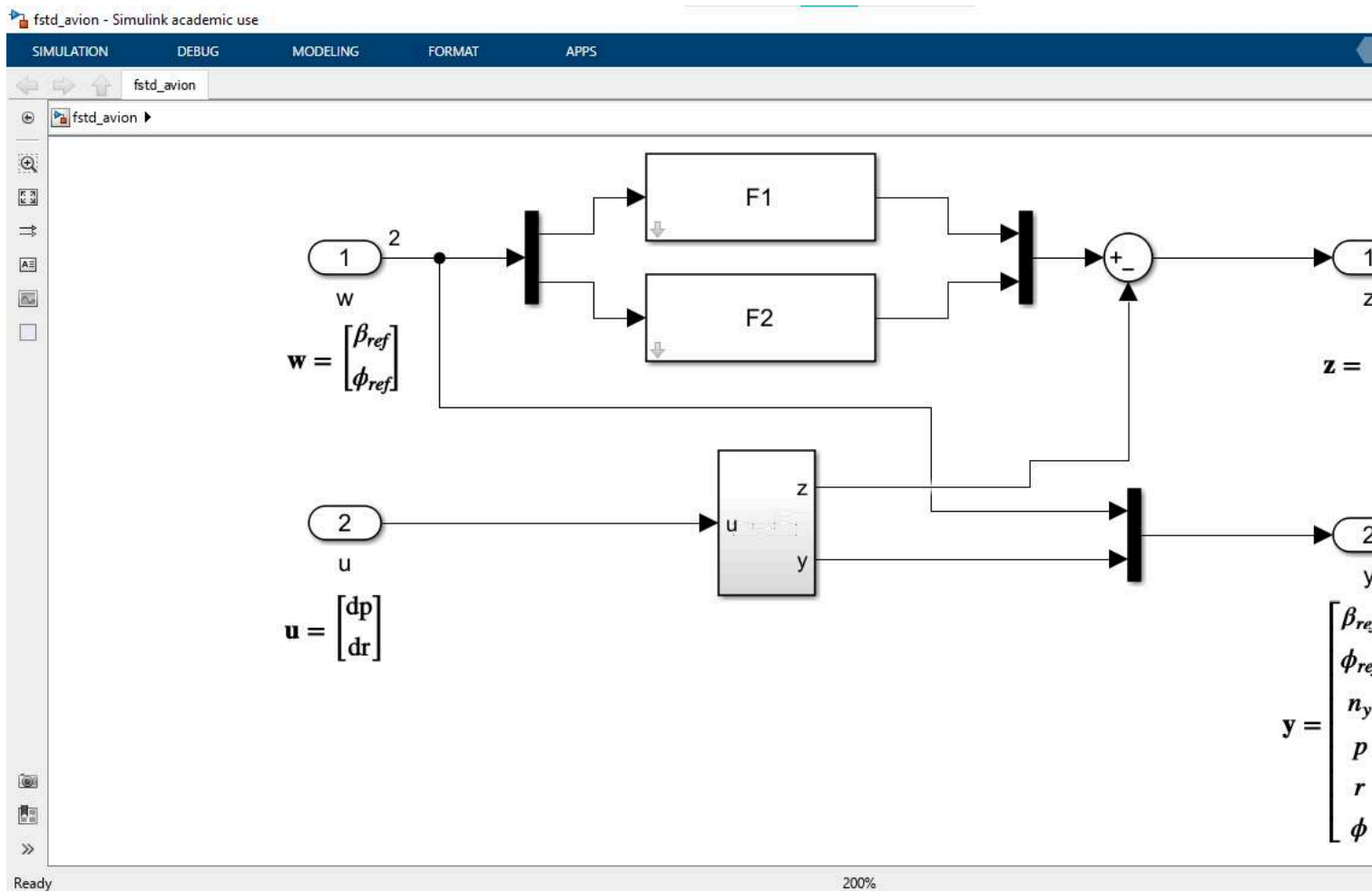
```
% step(Gbf(1:2,3:4))
H = inv(dcgain(Gbf(1:2,3:4)));
Gbf = linearize('CL1');
figure
step(Gbf(1:2,3:4))
```



On observe des performances similaires à celles obtenues par la méthode de placement de vecteurs propres. Le découplage est correct, on observe une légère oscillation de ϕ pour un échelon de β .

Contrôleur H_∞ à 2 degrés de liberté

On utilise le schéma simulink suivant:



Les filtres $F1$ et $F2$ sont choisis à partir des équations de découplage:

```
F1=tf(wl^2, [1 2*xil*wl wl^2]);
F2=tf(wr^2, [1 2*xir*wr wr^2]);
```

```
P= linearize('fstd_avion');
damp(P);
```

Pole	Damping	Frequency (rad/seconds)	Time Constant (seconds)
-3.84e-03	1.00e+00	3.84e-03	2.60e+02
-1.00e+00	1.00e+00	1.00e+00	9.97e-01
-1.03e+00	1.00e+00	1.03e+00	9.75e-01
-1.10e+00	1.00e+00	1.10e+00	9.12e-01
-1.85e-01 + 1.33e+00i	1.38e-01	1.34e+00	5.41e+00
-1.85e-01 - 1.33e+00i	1.38e-01	1.34e+00	5.41e+00
-1.00e+00 + 1.30e+00i	6.10e-01	1.64e+00	1.00e+00
-1.00e+00 - 1.30e+00i	6.10e-01	1.64e+00	1.00e+00

Synthèse H_∞ "full order":

```
[Kinf, CL, gamma]=hinfsyn(P,6,2);
gamma
```

```
gamma =
2.7462e-06
```

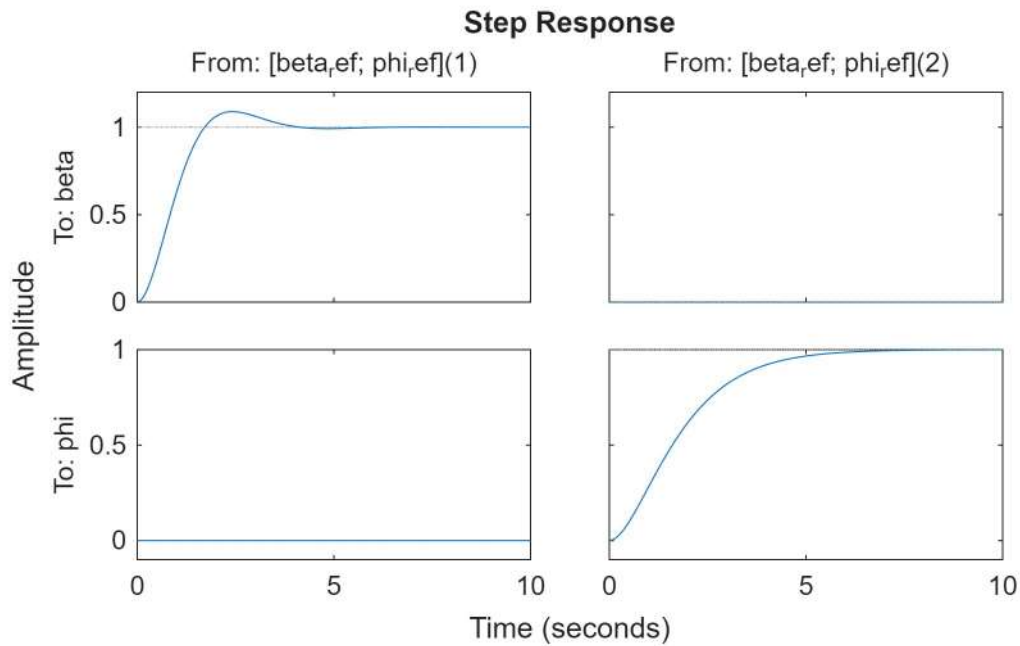
En régime permanent, l'erreur est inférieure ou égale à γ . La sortie suit donc quasi parfaitement le modèle de référence.

Analyse du controleur:

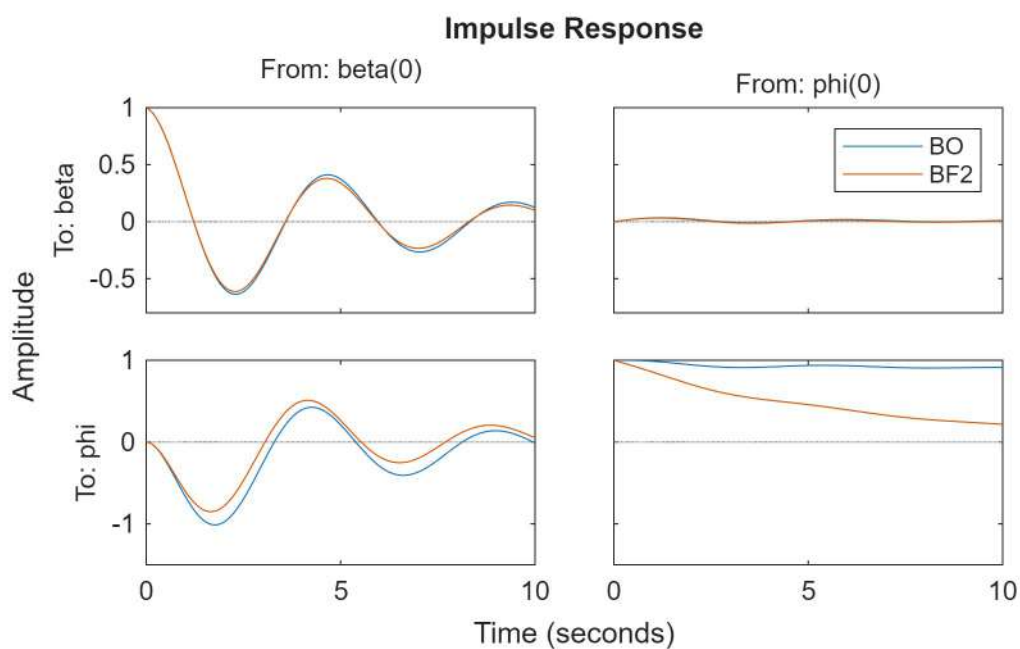
```
BF2=linearize('CL2');
```

Réponse à des échelons β_{ref} et ϕ_{ref} :

```
figure
step(BF2(1:2,3:4),10)
```



```
figure
impulse(B0(1:2,1:2),BF2(1:2,1:2),10); legend("BO", "BF2");
```



Le suivi de consigne est très bon, cependant le contrôleur ne rejette pas du tout les perturbations (conditions initiales sur β et ϕ) (comportement similaire à la boucle ouverte).

damp(Kinf)

Pole	Damping	Frequency (rad/seconds)	Time Constant (seconds)
-1.00e+00	1.00e+00	1.00e+00	9.97e-01
-1.10e+00	1.00e+00	1.10e+00	9.12e-01
-1.00e+00 + 1.30e+00i	6.10e-01	1.64e+00	1.00e+00
-1.00e+00 - 1.30e+00i	6.10e-01	1.64e+00	1.00e+00
-2.29e+01	1.00e+00	2.29e+01	4.36e-02
-4.58e+02 + 4.27e+02i	7.31e-01	6.26e+02	2.19e-03
-4.58e+02 - 4.27e+02i	7.31e-01	6.26e+02	2.19e-03
-7.44e+03	1.00e+00	7.44e+03	1.34e-04

Il y a dans le correcteur des modes très rapides qui poseraient problème lors de l'implémentation dans le calculateur de bord.

Synthèse H_∞ d'un contrôleur statique (systune)

Définition des paramètres à tuner:

```
Kinf = tunableGain("K2ddl",2,6); % Gain
CL0 = lft(P, Kinf);
CL0.u={'beta_ref','phi_ref'};
CL0.y={'d_beta','d_phi'};
```

Critère:

```
Req = TuningGoal.Gain({'beta_ref','phi_ref'},{'d_beta','d_phi'},1); % On
minimise l'erreur statique
```

Lancement

```
options = systuneOptions('MaxIter', 120000,'Display','sub','RandomStart',5);
[CLopt,fSoft,~,info] = systune(CL0,Req,[],options);
```

Design 1 of 6:
Final: Soft = 0.028, Hard = -Inf, Iterations = 118

Design 2 of 6:
Final: Soft = 0.028, Hard = -Inf, Iterations = 105

Design 3 of 6:
Final: Soft = 0.028, Hard = -Inf, Iterations = 94

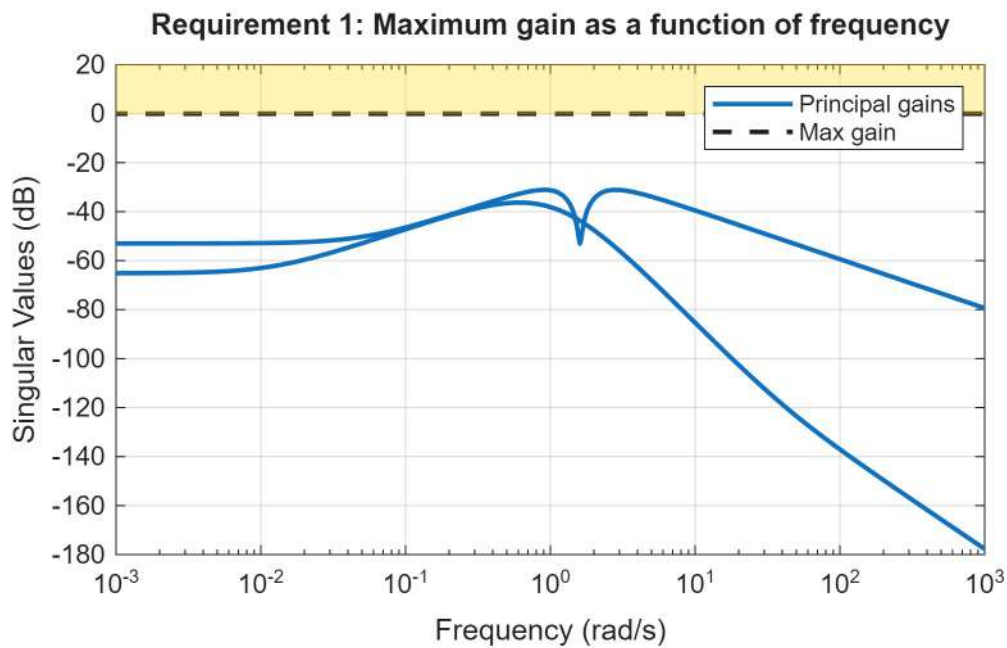
Design 4 of 6:
Final: Soft = 0.028, Hard = -Inf, Iterations = 107

Design 5 of 6:
Final: Soft = 0.028, Hard = -Inf, Iterations = 116

Design 6 of 6:
Final: Soft = 0.028, Hard = -Inf, Iterations = 130

figure

```
viewGoal(Req,Clopt)
```



Le critère fsoft est bien inférieur à 1. Le cahier des charges est bien respecté.

```
Kinf = getValue(Kinf,Clopt)
```

Kinf =

D =

	u1	u2	u3	u4	u5	u6
y1	1.385	2.659	124.6	-2.587	-0.2765	-2.701
y2	3.018	-0.05499	-20.52	-0.1116	1.764	-0.04036

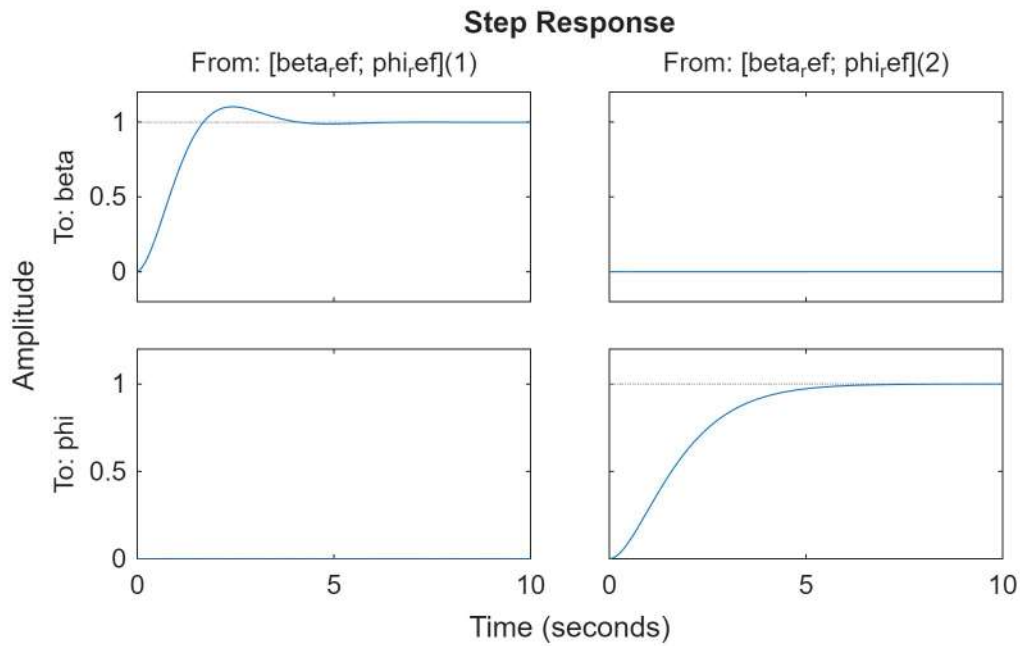
Name: K2ddl

Static gain.

Model Properties

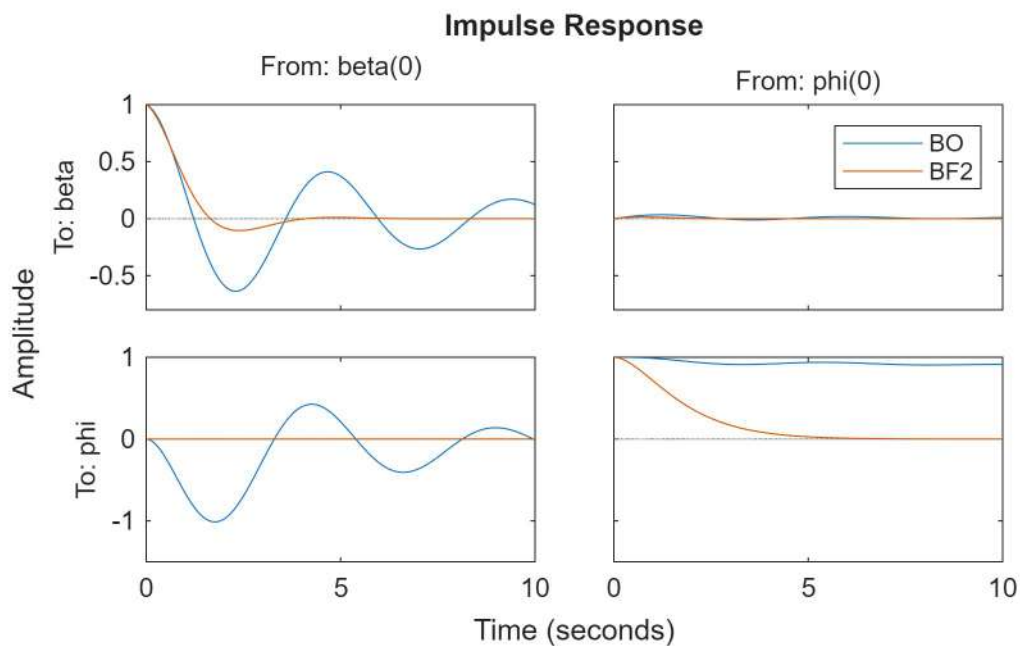
Analyse du contrôleur:

```
BF2=linearize('CL2');  
figure  
step(BF2(1:2,3:4),10)
```



Le suivi de consigne est bon, les états sont bien découplés.

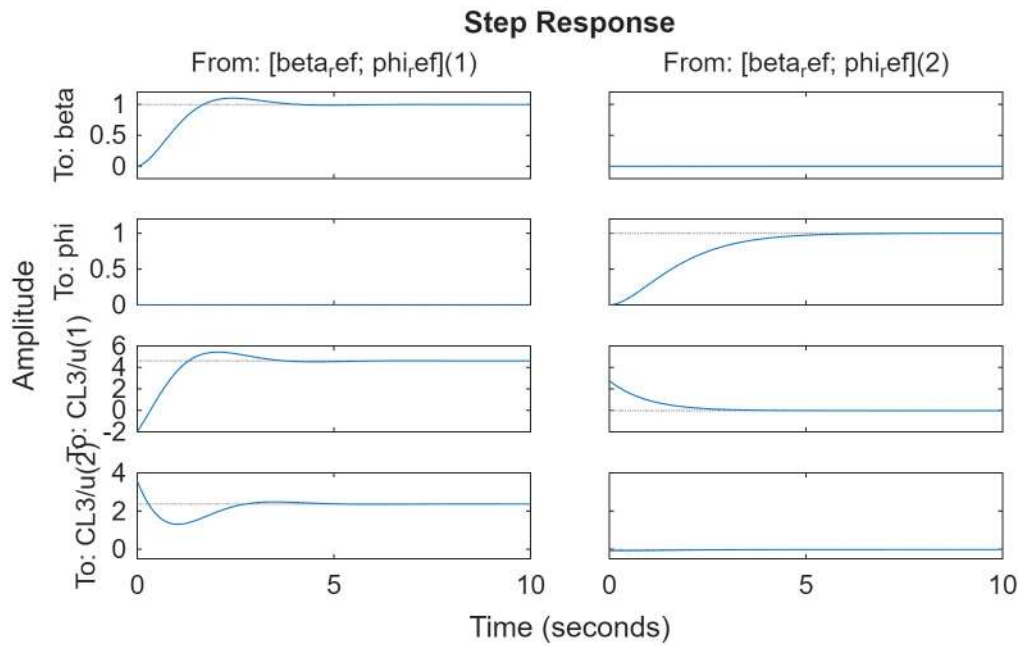
```
figure
impz(BO(1:2,1:2),BF2(1:2,1:2),10); legend("BO", "BF2")
```



Les perturbations sont bien rejetées, contrairement au full-order.

Réponse indicielle sur les braquages des gouvernes:

```
BF3 = linearize('CL3');
figure
step(BF3(:,3:4),10)
```

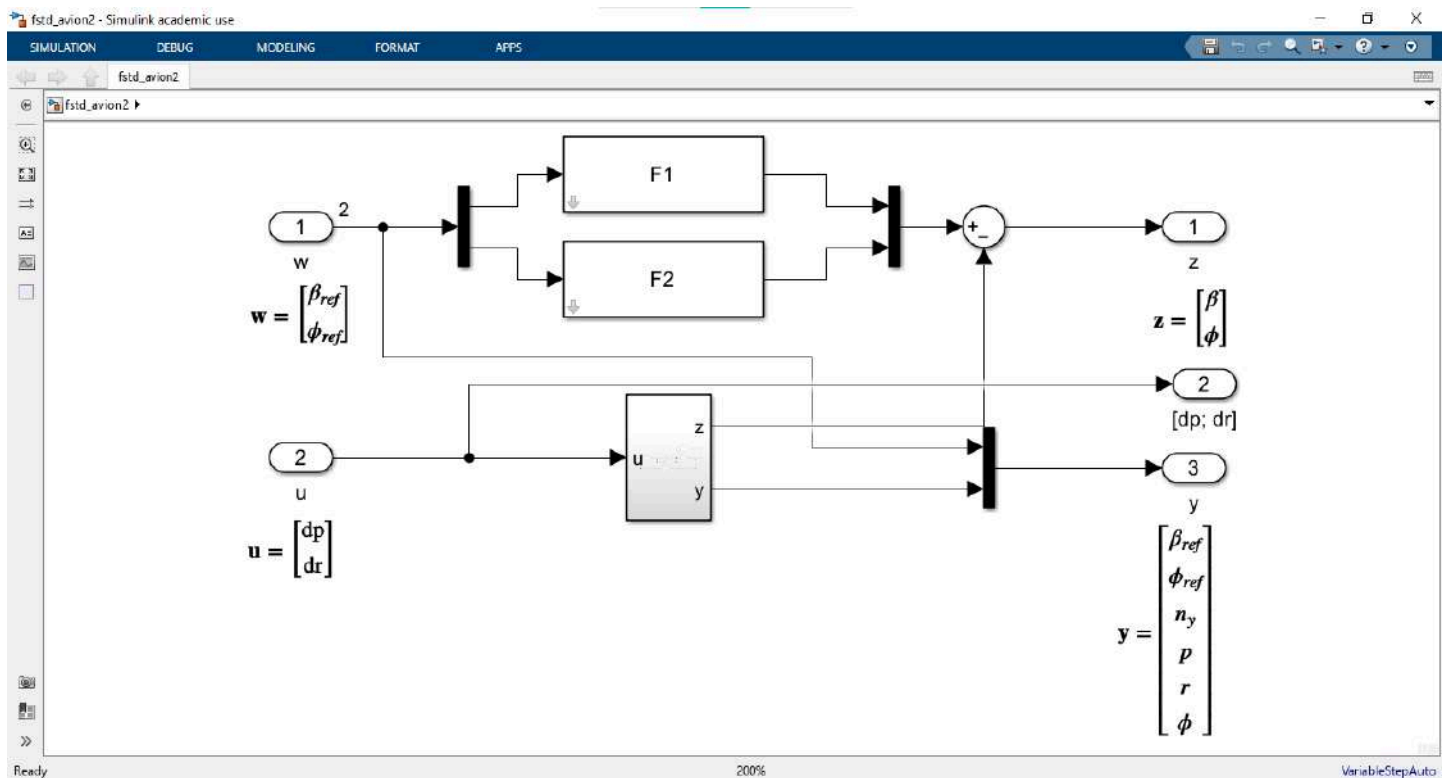


On observe que les deux gouvernes interviennent dans la correction des angles β et ϕ .

Synthèse H_2 d'un contrôleur statique

Dans ce cas, on fixe l'erreur à 0.05 (nous avons trouvé une solution de 0.028 à la partie précédente). On cherche alors un correcteur qui minimise l'énergie de commande (norme H_2).

Le nouveau problème standard de commande:



```
P= linearize('fstd_avion2')
```


P =

A =

	LTI System/I	LTI System/I	LTI System1/	LTI System1/	Integrator(1	Integrator(2	Integrator(
LTI System/I	-2.001	-1.345	0	0	0	0	
LTI System/I	2	0	0	0	0	0	
LTI System1/	0	0	-2.1	-1.1	0	0	
LTI System1/	0	0	1	0	0	0	
Integrator(1	0	0	0	0	-0.14	0.053	-0.99
Integrator(2	0	0	0	0	-2.461	-0.992	0.26
Integrator(3	0	0	0	0	1.595	-0.041	-0.26
Integrator(4	0	0	0	0	0	1	0.05

B =

	w(1)	w(2)	fstd_avion2/	fstd_avion2/
LTI System/I	1	0	0	0
LTI System/I	0	0	0	0
LTI System1/	0	1	0	0
LTI System1/	0	0	0	0
Integrator(1	0	0	0	0.03
Integrator(2	0	0	0.404	0.26
Integrator(3	0	0	0	-0.68
Integrator(4	0	0	0	0

C =

	LTI System/I	LTI System/I	LTI System1/	LTI System1/	Integrator(1	Integrator(2	Integrator(
fstd_avion2/	0	1.345	0	0	-1	0	
fstd_avion2/	0	0	0	1.1	0	0	
[dp; dr](1)	0	0	0	0	0	0	
[dp; dr](2)	0	0	0	0	0	0	
fstd_avion2/	0	0	0	0	0	0	
fstd_avion2/	0	0	0	0	0	0	
fstd_avion2/	0	0	0	0	0.0433	-0.0003	0.001
fstd_avion2/	0	0	0	0	0	1	
fstd_avion2/	0	0	0	0	0	0	
fstd_avion2/	0	0	0	0	0	0	

D =

	w(1)	w(2)	fstd_avion2/	fstd_avion2/
fstd_avion2/	0	0	0	0
fstd_avion2/	0	0	0	0
[dp; dr](1)	0	0	1	0
[dp; dr](2)	0	0	0	1
fstd_avion2/	1	0	0	0
fstd_avion2/	0	1	0	0
fstd_avion2/	0	0	0.0001	-0.0075
fstd_avion2/	0	0	0	0
fstd_avion2/	0	0	0	0
fstd_avion2/	0	0	0	0

Continuous-time state-space model.

Model Properties

```
Ksec = tunableSS("Ksec",2,2,6); % Gain
CL0 = lft(P, Ksec);
% CL0.u
% CL0.y
```

Critères:

```
Reqsoft = TuningGoal.Variance('w',[dp; dr],100);
Reqhard = TuningGoal.Gain('w','fstd_avion2/z',0.05); % Contrainte H_inf sur
l'erreur statique
```

Lancement

```
options = systuneOptions('MaxIter', 120000, 'Display', 'sub', 'RandomStart', 2);
[CLopt, fSoft, ~, info] = systune(CL0, Reqsoft, Reqhard, options);
```

Design 1 of 3:

```
alpha=0.1: Soft = 0.228, Hard = 0.7243, Iterations = 92
alpha=8.789: Soft = 0.112, Hard = 0.9907, Iterations = 61
alpha=17.78: Soft = 0.0921, Hard = 1.6381, Iterations = 60
alpha=13.34: Soft = 0.0996, Hard = 1.3284, Iterations = 31
alpha=11.11: Soft = 0.104, Hard = 1.1609, Iterations = 36
alpha=10: Soft = 0.107, Hard = 1.073, Iterations = 31
alpha=9.447: Soft = 0.109, Hard = 1.028, Iterations = 30
alpha=9.169: Soft = 0.109, Hard = 1.0276, Iterations = 11
alpha=9.03: Soft = 0.109, Hard = 1.0266, Iterations = 11
alpha=8.96: Soft = 0.109, Hard = 1.026, Iterations = 11
alpha=8.926: Soft = 0.109, Hard = 1.0252, Iterations = 11
alpha=8.908: Soft = 0.109, Hard = 1.0242, Iterations = 11
alpha=8.9: Soft = 0.109, Hard = 1.0239, Iterations = 11
Final: Soft = 0.112, Hard = 0.9907, Iterations = 407
```

Design 2 of 3:

```
alpha=0.1: Soft = 0.461, Hard = 0.86179, Iterations = 115
alpha=4.34: Soft = 0.157, Hard = 0.70486, Iterations = 26
alpha=12.71: Soft = 0.101, Hard = 1.2827, Iterations = 189
alpha=9.534: Soft = 0.109, Hard = 1.0352, Iterations = 48
alpha=7.945: Soft = 0.114, Hard = 0.91583, Iterations = 17
alpha=9.167: Soft = 0.11, Hard = 1.0053, Iterations = 35
alpha=8.984: Soft = 0.11, Hard = 1.0051, Iterations = 11
alpha=8.892: Soft = 0.11, Hard = 1.0046, Iterations = 11
alpha=8.846: Soft = 0.111, Hard = 0.97928, Iterations = 19
alpha=9.085: Soft = 0.11, Hard = 0.99845, Iterations = 26
alpha=9.115: Soft = 0.11, Hard = 1.0014, Iterations = 11
alpha=9.107: Soft = 0.11, Hard = 1.0013, Iterations = 11
Final: Soft = 0.11, Hard = 0.99845, Iterations = 521
```

Design 3 of 3:

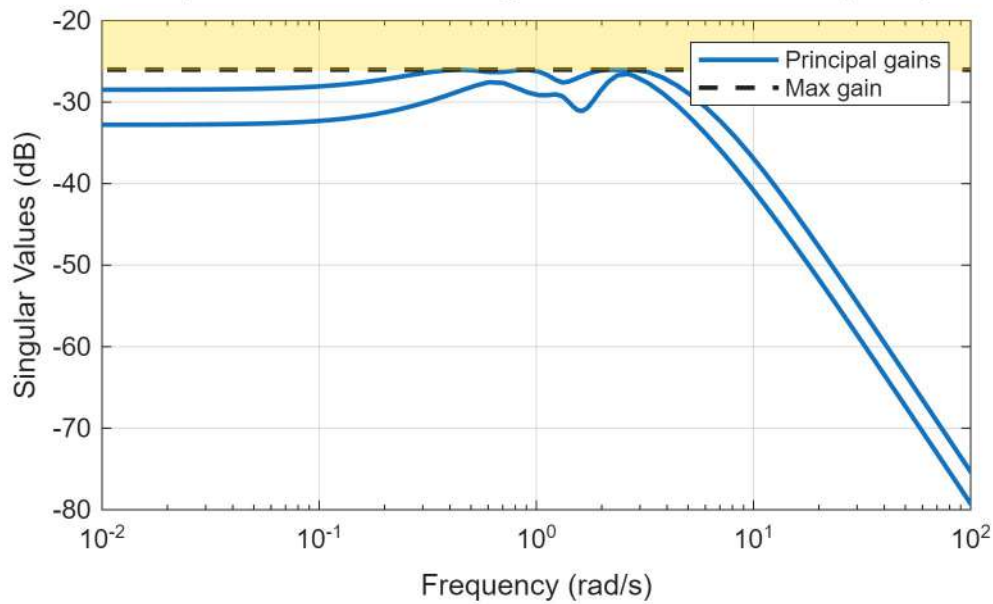
```
alpha=0.1: Soft = 0.568, Hard = 0.93086, Iterations = 96
alpha=3.522: Soft = 0.214, Hard = 0.79929, Iterations = 23
alpha=9.366: Soft = 0.109, Hard = 1.0217, Iterations = 121
alpha=7.024: Soft = 0.117, Hard = 0.86596, Iterations = 15
alpha=8.94: Soft = 0.11, Hard = 0.98766, Iterations = 25
alpha=9.209: Soft = 0.11, Hard = 1.0086, Iterations = 31
alpha=9.131: Soft = 0.11, Hard = 1.002, Iterations = 25
alpha=9.091: Soft = 0.11, Hard = 0.99888, Iterations = 16
alpha=9.116: Soft = 0.11, Hard = 1.0012, Iterations = 11
alpha=9.109: Soft = 0.11, Hard = 1.001, Iterations = 11
Final: Soft = 0.11, Hard = 0.99888, Iterations = 375
```

figure

```
% evalGoal(Reqsoft, CLopt)
```

```
viewGoal(Reqhard, CLopt)
```

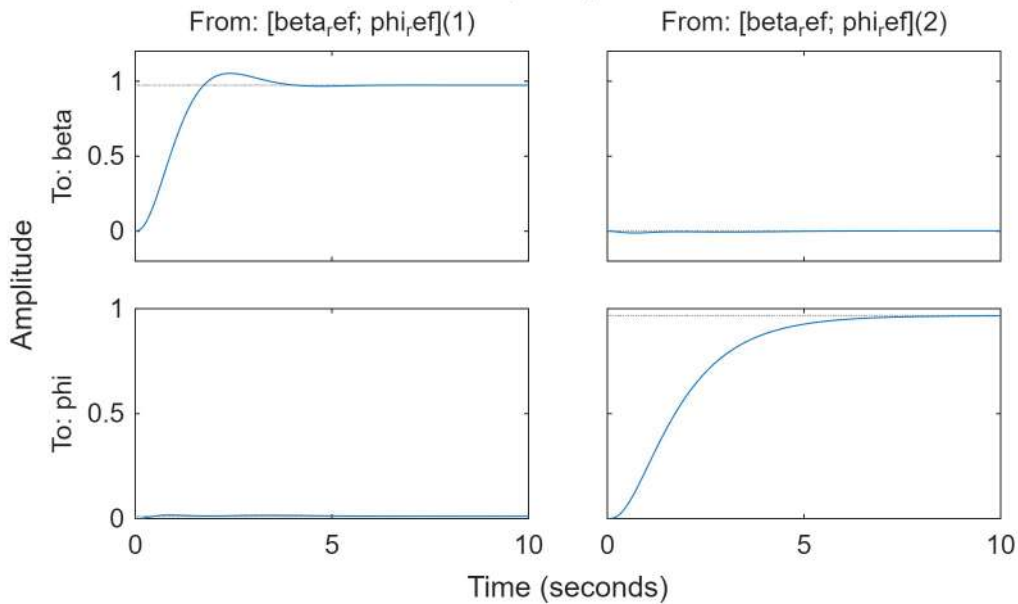
Requirement 1: Maximum gain as a function of frequency



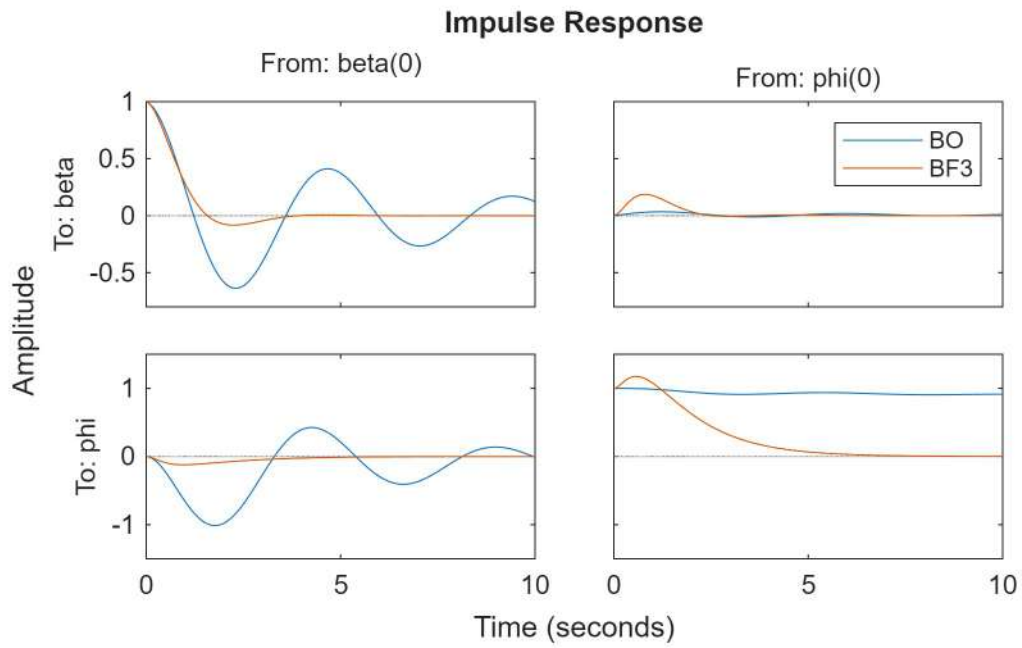
Le systune avec contrôleur d'ordre 2 trouve $\text{Hard} \approx 1.0$ (contrainte H^∞ active, juste satisfaite) et $\text{Soft} = 0.11$.

```
Kinf = getValue(Ksec,Clopt);
BF3 = linearize('CL3');
figure
step(BF3(1:2,3:4),10)
```

Step Response



```
impulse(B0(1:2,1:2),BF3(1:2,1:2),10); legend('B0','BF3');
```



La réponse indicielle et impulsionnelle montrent un bon compromis : le suivi est maintenu, le rejet de perturbation est amélioré par rapport à la BO, et l'énergie de commande est minimisée.